VesSkel

Vessel Skeletonization and Graph-Based Phenotype Analysis in Retinal Fundus Images

Simon Wittmann

Supervisor: Anna Möller

19. Mai 2026

Implementation Progress

- Experiment: Removing correlated features did not improve the best classifier
- Refactoring, better Docstrings and propagate plugin version to `napari.yaml`
- Unified `PipelineConfig` (JSON-serializable, shared napari + CLI)
- `vesskel`-CLI with `run` / `config-init` / `validate-config`
- Refactored napari widgets onto a single widget using shared `analyze_binary_image` pipeline
- Feature comparison table (REAVER, VesselVio, VesselExpress, TWOMBLI, VesSAP, Skan)

Unified Pipeline: napari + CLI

The core analysis is now a single shared function `analyze_binary_image()` used by both the napari widget and the new standalone CLI.

CLI usage: `vesskel run --input "data/*.png" --config config.json --out results/`

Config management: `vesskel config-init` (starter JSON), `vesskel validate-config` (check validity)

Config as contract: same file can be exported/imported inside napari

Current Limited Config-Options:

1. Branch extraction toggle, branch text labels, summary features, fractal dimension
2. Output controls: skeleton .npy/.png, summary.csv, per-image branches.csv

Feature Comparison: Tools Overview

See `Feature_Comparison.xlsx`

* I only took features that can be e2e-extracted using the Tool, e.g. Vessap uses their DL Network, then go through Allen Cell Registration (external Tool) and then analyze the Results using Matlab code.

Vipar is not included in the comparison: it is not an open-source package and cannot be used programmatically.

Skan is built around powerful pandas dataframes. Features are often not directly outputted but can be easily computed, e.g. `endpoint_count = np.sum(skel.degrees == 1)` or `min_segment_length = min(summarize()['branch_distance'])`. I treated features that can be easily computed as present.

This extensibility is a strong argument for further relying on the library.

Summary & Next Steps

What I did:

1. Unified pipeline
2. Standalone `vesskel` CLI (`run`, `config-init`, `validate-config`)
3. Configurable pipeline via JSON
4. Feature comparison table across 6 other tools

Next steps:

1. Add more features
2. Pipeline Performance Improvements?
3. Group Feature Table into Categories
4. Integrate original image for intensity-based features
5. Add Preprocessing-Options (Clique Removal, etc.)